



Guía de Uso Básico: Interfaz para Machine Learning

Integrado de Captura, Procesamiento y Predicción Artificial ("Matching Learning")

Proyecto: Captura de datos para machine learning

Carrera: Ingeniería Civil en Informática

Nombre: Paula Labra

Fecha: 26-05-2026

1. Introducción y Arquitectura del Directorio

Este tutorial proporciona el paso a paso para operar el programa de **Machine Learning**. El sistema permite hacer todo el proceso de forma ordenada: la captura de fotos con varias cámaras al mismo tiempo, organizar los datos en grupos, aplicar técnicas para generar más imágenes (data augmentation) y balanceo de datos, el entrenamiento de una red neuronal con Deep Learning y, finalmente, detectar o reconocer objetos en tiempo real con baja demora.

Para que el sistema funcione correctamente, es importante mantener el orden y la estructura de las tres carpetas principales del proyecto:

```
Matching Learning (Proyecto Raíz)
├── dataset/
│   ├── fondo_vacio/
│   ├── objeto_a/
│   └── dataset_final/
│       ├── train/
│       ├── val/
│       └── test/
├── modelos/
├── Rec.py
├── organizar_dataset.py
├── balancear_dataset.py
├── augment_data.py
└── entrenamiento.ipynb

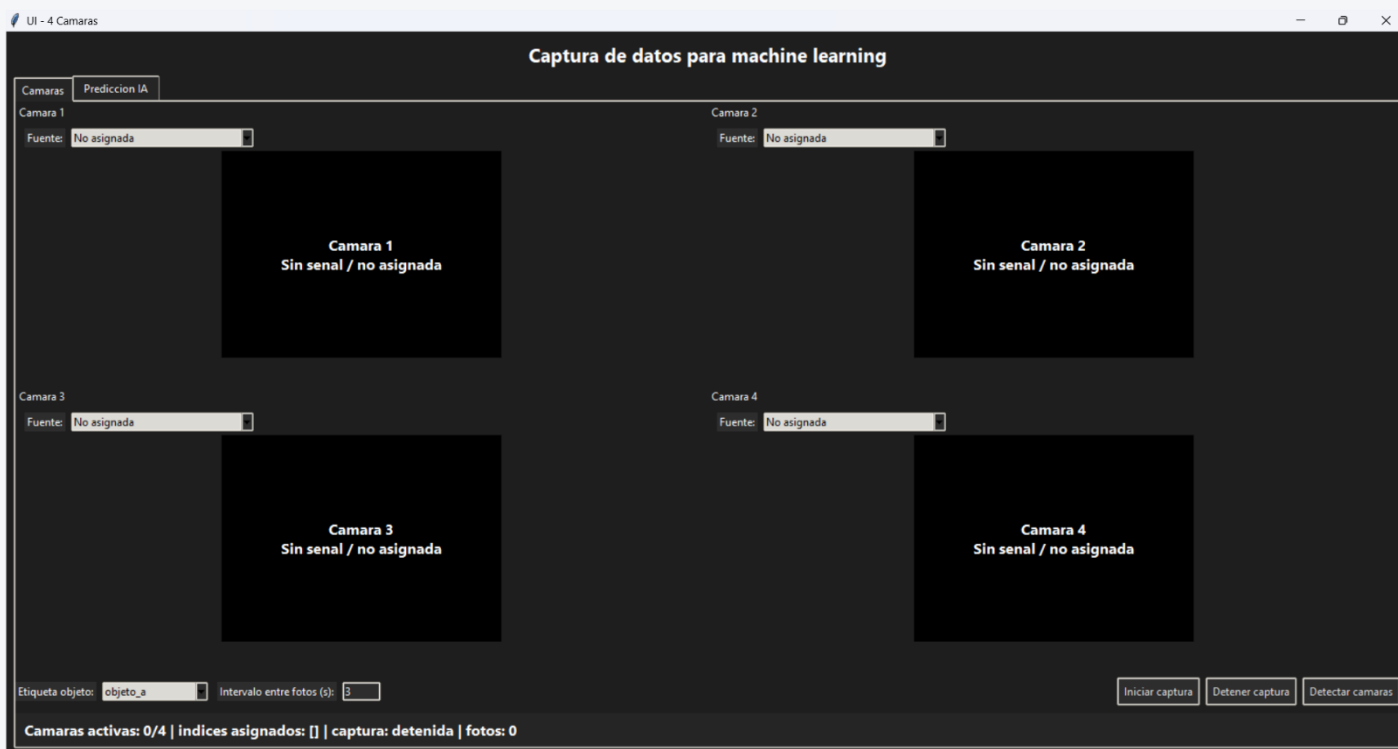
# Almacen de capturas directas desde la UI
# Capturas del entorno sin objeto
# Capturas del objeto objetivo
# Datos procesados, particionados y balanceados
# Subconjunto de entrenamiento
# Subconjunto de validación para ajuste
# Subconjunto de prueba ciega para evaluación final
# Repositorio de modelos serializados de Keras (.h5)
# Aplicación de interfaz GUI y lógica de captura
# Script de organización para los subconjuntos
# Script de sobremuestreo guiado por Albumentations
# Script complementario de expansión de imágenes
# Notebook de entrenamiento para Cloud TPU/GPU en Google
```

Carpetas	Función Principal	Dependencias Importantes
 dataset	Imágenes guardadas y organizadas según el nombre de cada etiqueta o categoría.	<code>cv2.imwrite</code> / Disparos síncronos
 dataset_final	Estructura organizada para que el sistema pueda cargar las imágenes durante el entrenamiento y las pruebas.	<code>tf.keras.utils.image_dataset_from_directory</code>
 modelos	Guardado de modelos entrenados.	TensorFlow HDF5 (<code>.h5</code>)

2. Desglose Teórico y Tutorial por Sección

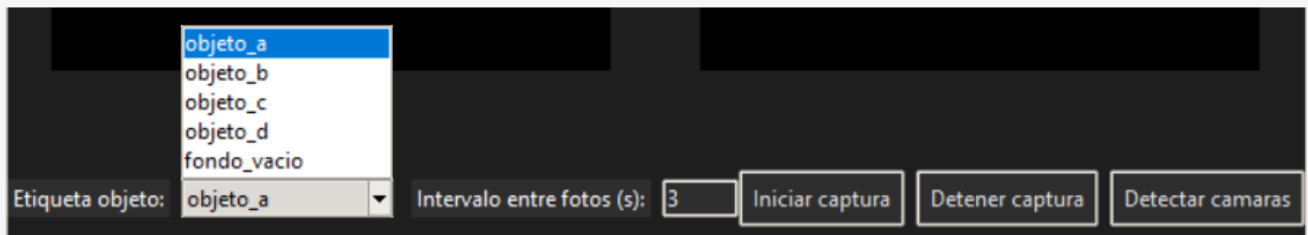
Sección A: Captura de datos en tiempo real e interfaz de captura (`Rec.py`)

Esta sección se encarga de conectar las cámaras con el programa. Utiliza **Tkinter** para la interfaz gráfica y **OpenCV** para comunicarse con las cámaras o webcams del sistema. Además, utiliza múltiples procesos al mismo tiempo para evitar que el programa se congele mientras se muestran las imágenes en pantalla.



- **Detección automática de cámaras:** El sistema busca y detecta automáticamente las cámaras conectadas y disponibles al computador. Es compatible con Windows y puede reconocer los nombres de los dispositivos de video.
- **Captura sincronizada (Matching Shot):** Cuando se usan varias cámaras al mismo tiempo (hasta 4), el sistema toma fotos de manera sincronizada. Cada captura recibe un número `disparo_id` único para identificarla y guardar las imágenes con un orden correcto `{disparo}_c{camara_index}.jpg` permitiendo relacionar fácilmente las fotos tomadas al mismo momento desde diferentes cámaras.

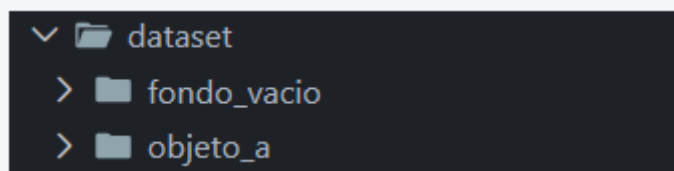
Una vez conocidas estas primeras funciones de la interfaz y estando en la pestaña principal, se tienen botones para la configuración de las capturas.



Donde:

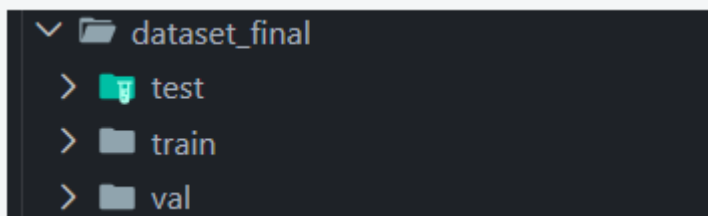
- **Etiqueta objeto:** Permite seleccionar o escribir el nombre de la categoría del objeto que será fotografiado. Esta etiqueta se utilizará para organizar automáticamente las imágenes capturadas dentro de la carpeta correspondiente.
- **Intervalo entre fotos (s):** Define el tiempo, en segundos, que el sistema esperará entre una captura y la siguiente. Esto permite controlar la frecuencia de adquisición de imágenes.
- **Iniciar captura:** Comienza el proceso de toma de fotografías utilizando las cámaras detectadas. El sistema empezará a capturar imágenes según el intervalo configurado.
- **Detener captura:** Finaliza el proceso de captura de imágenes en curso, deteniendo la toma automática de fotografías.
- **Detectar cámaras:** Busca y reconoce automáticamente las cámaras conectadas al equipo, permitiendo verificar qué dispositivos están disponibles para su uso, sin necesidad de reiniciar el programa.

Toda captura realizada se guarda en la carpeta dataset, que contendrá las fotografías de la etiqueta correspondiente.



Sección B: Partición Estadística (`organizar_dataset.py`)

Dividir las imágenes de forma aleatoria una por una podría causar problemas en los datos de varias cámaras. Esto ocurre porque fotos tomadas al mismo tiempo podrían quedar separadas entre los conjuntos de entrenamiento y validación. Como esas imágenes están relacionadas, el modelo podría aprender información que no debería conocer antes, generando resultados poco confiables o un sesgo en el entrenamiento.



Cuando elegimos organizar nuestro dataset, todos los archivos capturados son guardados automáticamente en la carpeta **dataset_final**. Dentro de esta carpeta, las imágenes se separan y organizan para el futuro entrenamiento.

Organización y separación de datos

En este lugar se agrupan los archivos según ID `disparo_id` de captura, asegurando que todas las fotos tomadas al mismo tiempo desde distintas cámaras se mantengan juntas. Luego, cada grupo completo de imágenes se asigna ya sea; `train` (70%), `val` (15%) o `test` (15%) como conjunto completo.

Sección C: Equilibrio de Clases y Creación de Nuevas Imágenes

Cuando una IA se entrena con muchas imágenes de una categoría y pocas de otra, puede aprender de forma desigual y dar mejores resultados solo para la clase que tiene más imágenes y malos para la otra clase. Para evitar este problema, el sistema utiliza herramientas que ayudan a equilibrar los datos, basado en la librería de alto rendimiento `Albumentations`

1. `augment_data.py` : Este módulo crea nuevas versiones de las imágenes actuales aplicando cambios automáticos, como giros, rotaciones, cambio de brillo o contraste y pequeñas variaciones visuales. Esto ayuda a que la IA aprenda mejor y pueda reconocer objetos incluso cuando las imágenes cambian de posición, iluminación y calidad.
2. `balancear_dataset.py` : En este archivo, el sistema revisa cuántas imágenes tiene cada categoría dentro del conjunto de entrenamiento. Si una clase tiene menos ejemplos, genera nuevas imágenes automáticamente hasta equilibrar la cantidad de datos entre ambas categorías.

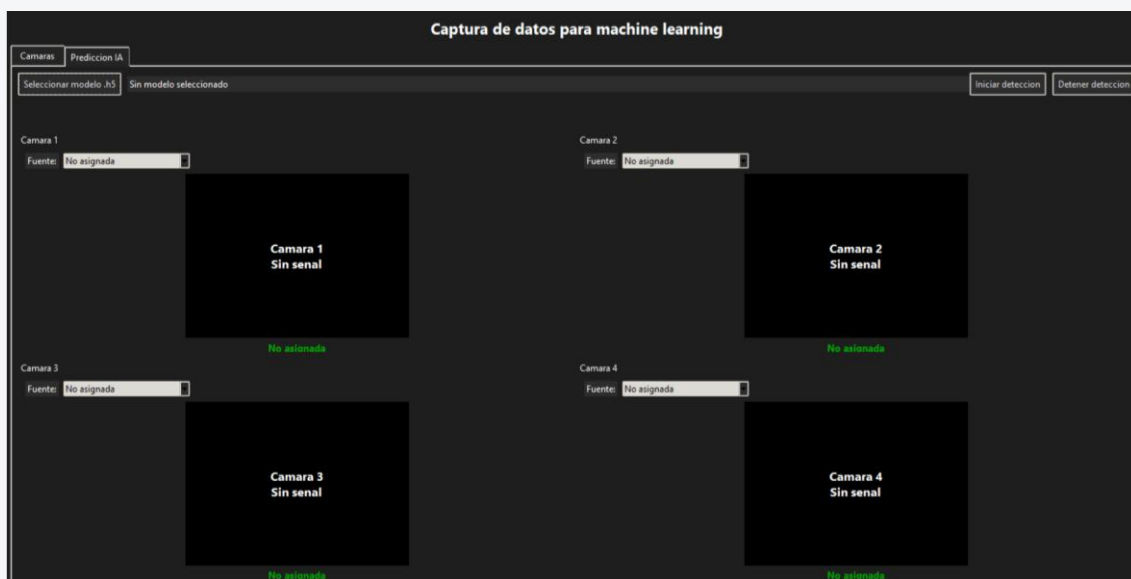
Sección D: Entrenamiento y Optimización del modelo (`entrenamiento.ipynb`)

Esta sección está pensada para ejecutarse fácilmente en plataformas como Google Colab. El sistema automatiza tareas como cargar archivos, descomprimir el dataset organizado (`dataset_final`) y preparar los datos para el entrenamiento del modelo, entregándonos un modelo terminado en `.h5`.

Además, utiliza técnicas para mejorar el aprendizaje de la IA y evitar errores por sobreentrenamiento (overfitting). Para ello, aplica métodos como dropout, que ayuda a que el modelo no dependa demasiado de ciertos patrones, Y detención temprana (early stopping), que detiene el entrenamiento cuando el rendimiento deja de mejorar. Finalmente, el sistema evalúa qué tan bien funciona el modelo utilizando métricas de rendimiento y una matriz de confusión, que permite ver con más detalle los aciertos y los errores de la IA.

Sección E: Pestaña de Predicción IA (`Rec.py`)

Esta sección del programa es donde ponemos a prueba a nuestra inteligencia artificial para ver si realmente aprendió a reconocer los objetos que le mostramos.



Aquí se despliegan las cámaras para identificar en tiempo real el objeto mostrado en cada pantalla. Una vez seleccionado el modelo .h5 desde la carpeta “modelos” e iniciamos la predicción, este modelo se activa y comienza a reconocer con un número encima de cada cámara. Lo llamamos score (o puntos de confianza) al número que calcula en pantalla junto con el nombre del objeto presentado.

La IA nunca está “100% segura” de nada de forma automática. Siempre calcula probabilidades. El sistema usa una escala matemática que va desde el 0.0 (que significa 0% de seguridad) hasta el 1.0 (que significa 100% de seguridad). El score ideal para que se sepa que el sistema reconoce el objeto es de 0.5, ya que relaciona lo que la IA ve con lo aprendido.



3. Flujo Tutorial Paso a Paso

PASO 1 Preparación del Entorno e Instalación de Dependencias

Abra su terminal del sistema o consola de comandos enfocada en la raíz del proyecto. Instale las librerías necesarias mediante el gestor de paquetes de Python de la siguiente manera:

```
pip install opencv-python tensorflow pillow albumentations pygrabber matplotlib seaborn
```

Asegúrese de comprobar que no existan conflictos en los controladores de video de sus cámaras antes de proceder al siguiente bloque. Si necesita la instalación de más dependencias o existe error en el código anterior, se agradece implementar o comentar la corrección para este tutorial.

PASO 2 Ejecución del Sistema de Captura **rec.py**

Ejecute la interfaz de usuario para inicializar el backend gráfico del sistema:

```
python Rec.py
```

Operaciones obligatorias dentro del panel de la UI:

- Diríjase a la pestaña llamada "**Cameras**".
- Haga clic en el botón de acción "**Detectar cámaras**". El log del sistema retornará los índices disponibles de video.
- Asigne las fuentes de las cámaras a los slots gráficos correspondientes en la interfaz.
- En la caja de texto o dropdown de etiquetas, ingrese de forma limpia la clase a capturar: primero ingrese `fondo_vacio` y realice capturas moviendo las cámaras o variando ligeras sombras; luego ingrese `objeto_a` y coloque el objeto a clasificar frente a los sensores ópticos.
- Pulse "**Iniciar captura**". El sistema guardará imágenes en `dataset/fondo_vacio` y `dataset/objeto_a` de manera automática y sincronizada.

PASO 3 Ejecución del Script de **organizar_dataset.py**

Una vez recolectadas suficientes imágenes crudas (mínimo sugerido de 200 disparos coordinados por clase), ejecute el archivo mencionado. Este proceso generará automáticamente la carpeta estructurada final:

```
python organizar_dataset.py --dataset "dataset" --salida "dataset_final" --seed 42
```

El script imprimirá en consola un resumen matemático exacto detallando cuántas imágenes de cada clase han sido asignadas exitosamente a los bloques de `train`, `val` y `test`.

PASO 4 Equilibrado `balancear_dataset.py`

Si se capturaron más imágenes de fondo que de objeto (un caso muy común), proceda a ejecutar el balanceador para equilibrar la cantidad de imágenes:

```
python balancear_dataset.py
```

Este comando analiza de forma interna el subdirectorio `dataset_final/train`, detecta qué clase posee menor cantidad de archivos e inyecta imágenes generadas mediante Albumentations hasta lograr una proporción simétrica.

PASO 5 Entrenamiento del Modelo en `entrenamiento.ipynb`

Suba la carpeta completa creada como `dataset_final` a su almacenamiento en la nube de Google Drive, o empaquétele en un formato comprimido llamado (`dataset_final.zip`).

- Activa la aceleración por hardware seleccionando GPU o TPU en el entorno de ejecución.
- Ejecuta las celdas del notebook en orden. El notebook cargará automáticamente los datos, los preparará para el entrenamiento y construirá la red neuronal utilizando el optimizador Adam.
- Cuando termine el entrenamiento, se generará y guardará el modelo en la ruta `modelos/modelo_buho_v1.h5`.
- Descarga ese archivo y guárdalo en la carpeta **modelos/** de tu proyecto para utilizarlo posteriormente

PASO 6 Iniciar predicción en tiempo real

Para visualizar los resultados del modelo predictivo en tiempo real, ejecute la interfaz local nuevamente:

```
python Rec.py
```

Módulo de Predicción IA:

1. Navegue directamente hacia la segunda pestaña de la interfaz llamada como **"Predicción IA"**.
2. Haga clic en el botón **"Seleccionar modelo .h5"** y busque en su explorador de archivos el nombre del modelo creado. (Por ej. dentro de la carpeta `modelos/modelo_buho_v1.h5`).
3. Haga clic en el botón **"Iniciar detección"**.

Nota Importante

El sistema analizará continuamente el video de la cámara utilizando el modelo entrenado. En pantalla se mostrará la predicción realizada y el porcentaje de confianza correspondiente en tiempo real

4. Desafío

En este desafío, en base a todo el documento y código entregado, se pide realizar un programa en Python con la finalidad de adaptar la aplicación a un "robot móvil" con dos cámaras.

Así las cámaras "miran" hacia el frente. La idea es reconocer elementos arquitectónicos, personas, veredas, de tal manera que el robot pueda identificarlos, y en tiempo "real" realice los cambios o dirección de la trayectoria.

¡Buena suerte!